Parameter Sharing Is Not Compute Sharing

A Matched-Cost Study of Fractal / Recursive Routing at ≤10M Parameters

Abstract. We built a routing architecture whose compute flows *through depth* via an intention vector — one router, one halt head, one governance policy applied identically at every scale, with experts cloned from hot slots and retired from cold ones — and asked whether it can beat a dense transformer at **matched inference cost**. It cannot, on this task, and the reason is structural rather than a tuning accident. Three results fix the verdict. (1) **Dominance:** across five post-fix fractal configurations spanning 1.9M–8.5M parameters and 21M–132M MACs/item, for every point there exists a dense model at *lower* cost with *higher* accuracy — 5 of 5. (2) **The compute does not shrink:** weight tying cuts parameters by 87% (8.53M → 1.10M at d=256/L=8) while leaving MACs/item *exactly* unchanged, and costs 23% relative accuracy; parameter sharing is not compute sharing. (3) **The headroom does not exist:** we computed the *oracle* bound on per-item adaptive depth — the best possible depth choice for every difficulty bucket, given the model's own measured accuracy-vs-depth curve. The oracle's maximum attainable accuracy equals the best fixed depth to two decimal places (**61.87% vs 61.87%, gain +0.00 points**), and extra depth actively *hurts* (depth 8 < depth 6). Adaptive-depth routing cannot recover a gap that a perfect depth oracle cannot even see. Meanwhile the router itself is healthy: gradient reaches both the hidden state (grad_h ≈ 5–10e-5) and the logits through the softmax Jacobian (grad_z ≈ 5e-6), N_eff/E = 0.993–0.997, zero dead slots. **The routing works. Routing is simply not where the compute is.** We also correct a measurement error that made our own earlier "dense wins" claims too generous to us: cost must be per *item*, and generated-token length is identical across architectures, so trace-shortening saves 33% for both families at once.

Budget note. Every row was trained for the number of steps named in its caption (3,000 or 9,000) with one shared optimizer and schedule; §6 shows why that matters (+12.2 points from training length alone). Accuracy is best-by-exact-match for both families. Cost is MACs per **item**, using each dataset's own measured mean sequence length.

---

1. The claim under test

The pitch for a "fractal" or recursive transformer: instead of stacking *different* layers, apply the *same* cell repeatedly, route through a persistent intention vector, and let easy tokens exit early. Then one cell buys you arbitrary depth at O(1) parameter cost, and adaptive depth buys you compute back.

Two separate claims are hiding in that sentence, and separating them is most of this paper:

- **Parameter claim:** reuse one cell → far fewer parameters for the same depth.
- **Compute claim:** route tokens through fewer levels → far fewer FLOPs at the same accuracy.

We find the first claim **true and nearly free**, and the second **false on this task** — not "hard to achieve", but provably unavailable, because the accuracy-vs-depth curve of a trained model has no slack for a depth policy to exploit.

2. What we built

`src/hmoe/model.py`, ~850 lines, character-level arithmetic:

- **`IntentionRouter`** — `z = T · normalize(proj(LayerNorm(h))) + bias`. Logit RMS is *pinned* by construction (`u/rms(u)·logit_rms`) so the router cannot saturate or deafen itself; a `bias` buffer

carries non-differentiable load balancing that the softmax's shift-invariance would otherwise absorb.

- **`Cell`** — attention, an intention update `h += h_mlp([ln(a), ln(h)])`, and a routed mixture of `E` SwiGLU experts. `ln_expert` normalises the expert input (§ Paper 2).
- **Per-token depth routing.** Let `c_l(t) = σ(halt(h_l))` be "continue past level l" and `s_l(t) = Π_{j<l} c_j(t)` survival. The cell delta is gated by survival:

```
a  = x + s·attn(x);   h = h + s·h_mlp([a,h]);   x' = LN(a + s·mix(h)(a))
```

  Once `s → 0` the token stops changing *entirely*, so its LayerNorm'd k/v freeze and inference can skip it exactly — trained function and executed function coincide, no adaptive-computation mismatch. Cost is charged with a surrogate of the **executor's rule**, not textbook ACT: `D = 1 + Σ_l σ((Σ_{j<l} log c_j − log τ)/κ)`, so trained cost and paid cost converge together.
- **Pool governance** (`govern()`) — cold slots (load window below threshold for `k_cold` consecutive checks) are overwritten with a *perturbed copy* of a hot expert (`rho`-scaled weight noise), their router bias reset to 0, load counters zeroed. Cohesion is measured, not assumed (§7).
- **`share_weights`** — one cell reused at every level (strict fractal) or one cell per level.

Two-stage training is mandatory: a depth gate free to close from step 0 **always closes** (measured: executed depth fell to 1.94 with exact match pinned at 12%). So stage 1 trains full-depth with deep supervision (`--no-depth-gate --deep-coef 0.3`), and only afterwards is an exit calibrated.

3. Cost accounting, done honestly

Three rules, each of which we initially violated:

1. **Per item, not per token.** A model that emits shorter answers is cheaper. MACs/item = MACs/token × mean(prompt+target tokens) measured from the split's own metadata (§A). Assuming a sequence length silently changed conclusions.
2. **Name the dataset in every row.** Comparing a model trained on 44-token items against one trained on 30-token items is not a comparison.
3. **Fix the training budget.** See §6.

Cost is analytic (`macs_per_token`), counting attention as `6·d² + 2·d·T` per token and top-1 gathered experts as one expert's FFN, which matches the executor.

4. Result 1 — dense dominates at every matched cost point

Dense frontier (8 layers/ffn128 reference is `REF9k_L8F128`; all 9,000 steps, `data/calc_mix`):

run	params	levels	MACs/item	exact	answer
R9_d128_L4F128	481,920	4	20,332,055	59.7%	60.3%
R9_d128_L4F256	678,528	4	29,043,362	**67.8%**	68.3%
R9_L2F192	863,488	2	36,308,456	62.0%	62.5%
R9_L4F128	1,488,128	4	63,894,263	71.8%	72.2%
REF9k_L8F128	2,934,016	8	127,777,183	**73.2%**	74.1%

Post-fix fractal frontier (same data, same 9,000 steps):

run	params	levels	MACs/item	exact	answer
FIX_d128_e128_f4	1,885,957	4	21,472,011	58.6%	60.1%
FIX_d128_e128_f6	2,818,183	6	32,205,181	60.2%	61.2%
LNE_D128_long	3,750,409	8	42,938,351	60.0%	61.1%
FIX_d128_e256_f8	6,896,137	8	60,360,966	60.8%	62.1%
FIX_d256_e128_f8	8,531,081	8	131,508,980	68.6%	69.2%

The dominance test. For each fractal point, the best dense model at or below its cost:

fractal	MACs/item	exact	best dense ≤ that cost	dense MACs/ item	dense exact	verdict
FIX_d128_e128_f4	21,472,011	58.6%	R9_d128_L4F128	20,332,055	59.7%	**dense wins**
FIX_d128_e128_f6	32,205,181	60.2%	R9_d128_L4F256	29,043,362	67.8%	**dense wins**
LNE_D128_long	42,938,351	60.0%	R9_d128_L4F256	29,043,362	67.8%	**dense wins**
FIX_d128_e256_f8	60,360,966	60.8%	R9_d128_L4F256	29,043,362	67.8%	**dense wins**
FIX_d256_e128_f8	131,508,980	68.6%	REF9k_L8F128	127,777,183	73.2%	**dense wins**

5 of 5. The fractal family is not “slightly behind”; it is *Pareto-dominated* — there is no cost level at which you should prefer it. Note the shape: dense accuracy rises steeply with FFN width (67.8% at 29M MACs), while the fractal plateaus near 60–61% from 32M to 60M MACs and only moves when *width* d goes 128→256, where it is paying for dense-like attention anyway.

Scope of this document (added by the follow-up study)

Everything above concerns **one cell applied repeatedly along depth** (weight tying across depth, i.e. self-similarity across *step*). It does not test scale-recursive routing — experts that are themselves routed cells, self-similar across *scale* — and its conclusions should not be extended to that architecture. The scale-recursive case is tested, at matched parameters / active parameters per token / MACs-per-item, in `research/PAPER_3_scale_recursive_routing.md`. That study leaves this document unchanged and cites it as its Condition 2 control.